CS486/686: Introduction to Artificial Intelligence
Lecture 9c - Reinforcement Learning

Jesse Hoey & Victor Zhong

School of Computer Science, University of Waterloo

March 27, 2025

Readings: Poole & Mackworth Chap. 13.1-13.9

Reinforcement Learning

What should an agent do given:

- **Prior knowledge**: possible states of the world
  possible actions
- **Observations**: current state of world
  immediate reward / punishment
- **Goal**: act to maximize accumulated reward

Like decision-theoretic planning, except model of dynamics and model of reward not given.

Experiences

- We assume there is a **sequence of experiences**:

$$\textit{state}, \textit{action}, \textit{reward}, \textit{state}, \textit{action}, \textit{reward}, \ldots.$$

- What should the agent do next?
- It must decide whether to:
  - **explore** to gain more knowledge
  - **exploit** the knowledge it has already discovered

Reinforcement Learning: "Bandit" problem

Each machine has a $Pr(win)$ ... but you don't know what it is...

Which machine should you play?

Why is reinforcement learning hard?

- **credit assignment** problem: what actions are responsible for the reward may have occurred a **long time before** the reward was received.
- The **long-term effect** of an action of the robot depends on what it will do in the future, but it does not have a model to predict this.
- The **explore-exploit dilemma**: at each time should the robot be greedy or inquisitive?

Reinforcement learning: main approaches

- **Model-Free RL learn** $Q^*(s, a)$, use this to guide action.
- **Model Based RL**: **learn a model** consisting of state transition function $P(s'|a, s)$ and reward function $R(s, a, s')$; **solve** this as an MDP.
- search through a space of policies

Temporal Differences

- Suppose we have a sequence of values:

$$v_1, v_2, v_3, \ldots$$

And want a **running estimate** of the average of the first k values:

$$A_k = \frac{v_1 + \cdots + v_k}{k}$$

Temporal Differences (cont)

- When a new value v_k arrives:

$$\begin{aligned} A_k &= \frac{v_1 + \cdots + v_{k-1} + v_k}{k} \\ kA_k &= v_1 + \cdots + v_{k-1} + v_k \\ &= (k-1)A_{k-1} + v_k \\ A_k &= \frac{k-1}{k} A_{k-1} + \frac{1}{k} v_k \end{aligned}$$

  Let $\alpha = \frac{1}{k}$, then

$$\begin{aligned} A_k &= (1-\alpha)A_{k-1} + \alpha v_k \\ &= A_{k-1} + \alpha(v_k - A_{k-1}) \end{aligned}$$

  "TD formula"

- Often we use this update with α **fixed.**

Q-learning

- **Idea:** store $Q[State, Action]$; update this as in **asynchronous value iteration**, but using experience (empirical probabilities and rewards).
- Suppose the agent has an experience $\langle s, a, r, s' \rangle$
- This provides one piece of data to update $Q[s, a]$.
- The experience $\langle s, a, r, s' \rangle$ provides the data point:

$$r + \gamma \max_{a'} Q[s', a']$$

which can be used in the **TD formula** giving:

$$Q[s, a] \leftarrow Q[s, a] + \alpha \left(r + \gamma \max_{a'} Q[s', a'] - Q[s, a] \right)$$

Q-learning

initialize $Q[S, A]$ arbitrarily
observe current state s
repeat forever:
  select and carry out an action a
  observe reward r and state s'
  $Q[s, a] \leftarrow Q[s, a] + \alpha \left(r + \gamma \max_{a'} Q[s', a'] - Q[s, a]\right)$
  $s \leftarrow s'$;
end repeat

Properties of Q-learning

- Q-learning **converges to the optimal policy**, no matter what the agent does, as long as it **tries each action in each state enough (infinitely often).**
- But what should the agent do?
  - **exploit**: when in state s, select the action that maximizes $Q[s, a]$
  - **explore**: select another action

Exploration Strategies

- The ϵ-**greedy** strategy: choose a random action with probability ϵ and choose a best action with probability $1 - \epsilon$.
- **Softmax** action selection: in state s, choose action a with probability

$$\frac{e^{Q[s,a]/\tau}}{\sum_a e^{Q[s,a]/\tau}}$$

where $\tau > 0$ is the **temperature**. Good actions are chosen more often than bad actions; τ defines how often good actions are chosen. For $\tau \to \infty$, all actions are equiprobable. For $\tau \to 0$, only the best is chosen.

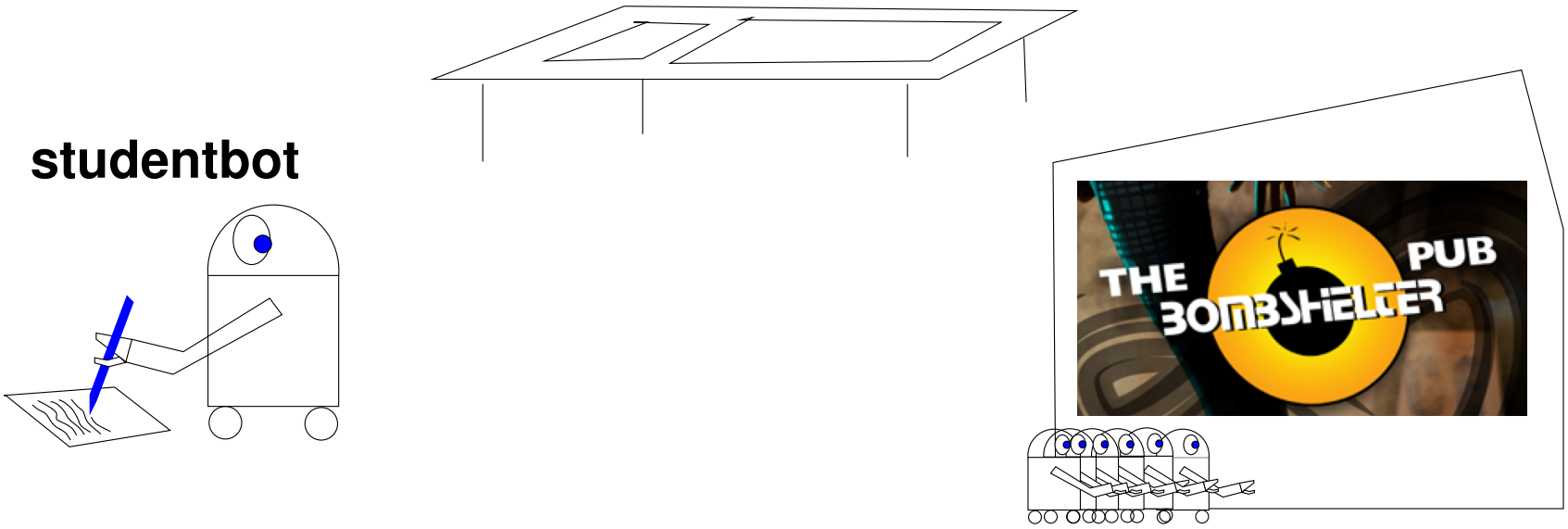
Exploration Strategies

- **optimism in the face of uncertainty**: initialize Q to values that encourage exploration.
- **Upper Confidence Bound (UCB)**: Also store $N[s, a]$ (number of times that state-action pair has been tried) and use

$$\arg\max_a \left[Q(s,a) + k\sqrt{\frac{N[s]}{N[s,a]}} \right]$$

where $N[s] = \sum_a N[s,a]$

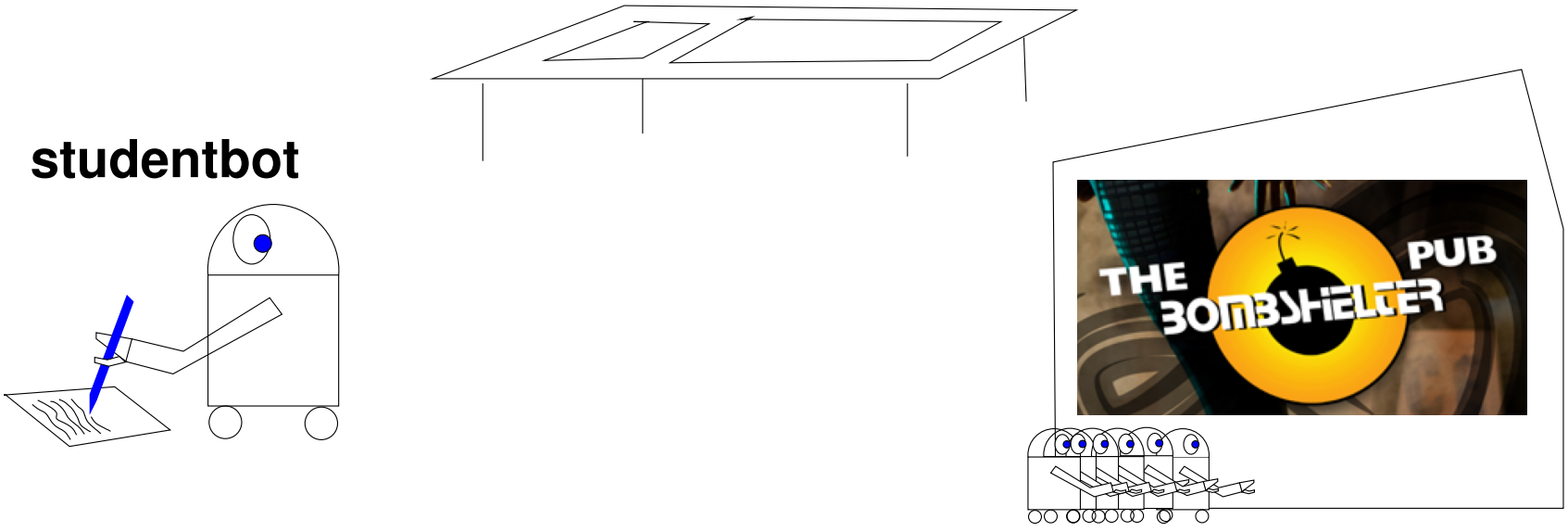
Example: Studentbot



state variables:

- **tired**: studentbot is tired (no/a bit/very)
- **passtest**: studentbot passes test (no/yes)
- **knows**: studentbot's state of knowledge (nothing/a bit/a lot/everything)
- **goodtime**: studentbot has a good time (no/yes)

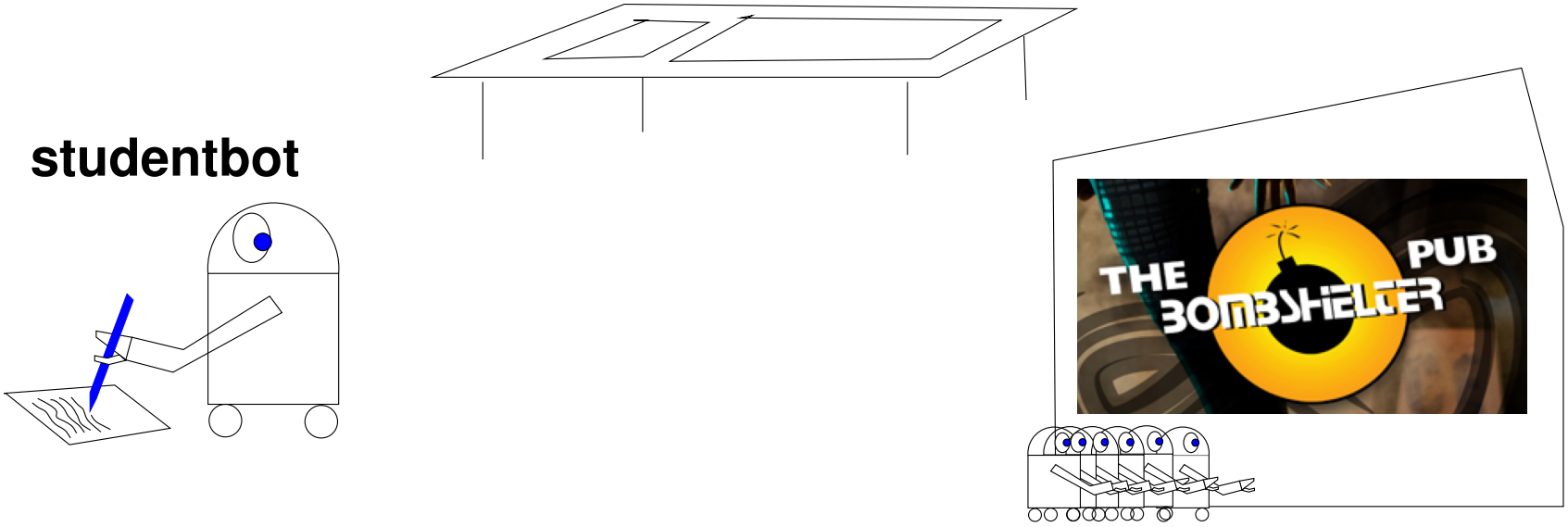
Example: Studentbot



studentbot actions:

- **study**: studentbot's knowledge increases, studentbot gets tired
- **sleep**: studentbot gets less tired
- **party**: studentbot has a good time, but gets tired and loses knowledge
- **take test**: studentbot takes a test (can take test anytime)

Example: Studentbot



studentbot rewards:

- **+20** if studentbot passes the test
- **+2** if studentbot has a good time when not very tired

basic tradeoff: short term vs. long-term rewards

Studentbot Policy

tired	know	policy	value
no	nothing	party	10.53
no	a bit	party	11.96
no	a lot	sleep	18.05
no	everything	take test	33.73
a bit	nothing	sleep	9.47
a bit	a bit	take test	10.02
a bit	a lot	sleep	27.54
a bit	everything	take test	32.39
very	nothing	sleep	9.52
very	a bit	sleep	11.08
very	a lot	sleep	27.52
very	everything	take test	20.47

Policy after 100,000 RL steps for pass=no, gt=no

policy	value
study	16.7
study	20.8
study	25.7
take test	31.6
study	15.97
study	19.94
study	26.0
take test	31.6
sleep	15.1
sleep	18.7
sleep	23.1
sleep	28.4

Optimal Value and Policy

Studentbot Policy

tired	know	policy	value
no	nothing	party	10.53
no	a bit	study	14.61
no	a lot	study	25.3
no	everything	take test	29.9
a bit	nothing	take test	9.52
a bit	a bit	take test	16.06
a bit	a lot	sleep	21.43
a bit	everything	sleep	39.8
very	nothing	sleep	9.47
very	a bit	sleep	13.41
very	a lot	sleep	17.8
very	everything	sleep	28.32

Policy after 200,000 RL steps for pass=no, gt=no

policy	value
study	16.7
study	20.8
study	25.7
take test	31.6
study	15.97
study	19.94
study	26.0
take test	31.6
sleep	15.1
sleep	18.7
sleep	23.1
sleep	28.4

Optimal Value and Policy

Off/On-policy Learning

- Q-learning does **off-policy** learning: it learns the value of the optimal policy, no matter what it does.
- This could be bad if the exploration policy is dangerous.
- **On-policy** learning learns the value of the policy being followed. e.g., act greedily 80% of the time and act randomly 20% of the time
- If the agent is actually going to explore, it may be better to optimize the actual policy it is going to do.
- **SARSA** uses the experience $\langle s, a, r, s', a' \rangle$ to update $Q[s, a]$.
- **SARSA** is **on-policy** because it uses empirical values for s' and a'

SARSA

initialize $Q[S, A]$ arbitrarily
observe current state s
select action a using a policy based on Q **includes exploration**
repeat forever:
  carry out an action a
  observe reward r and state s'
  select action a' using a policy based on Q
  $Q[s, a] \leftarrow Q[s, a] + \alpha \left(r + \gamma Q[s', a'] - Q[s, a]\right)$
  $s \leftarrow s'$;
  $a \leftarrow a'$;
end-repeat

Model-based Reinforcement Learning

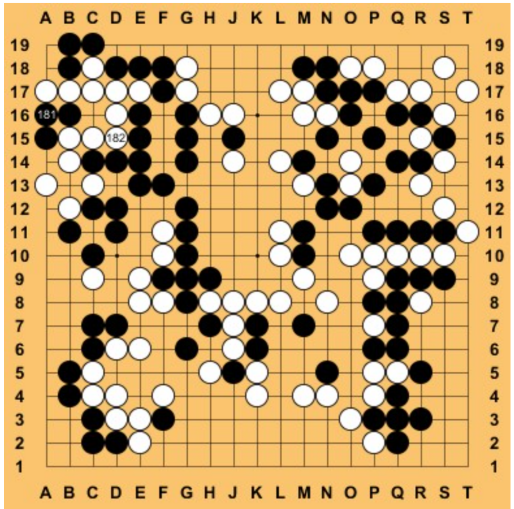
- **Model-based** reinforcement learning uses the experiences in a more effective manner.
- It is used when **collecting experiences is expensive** (e.g., in a robot or an online game), and you can do lots of computation between each experience.
- **Idea**: learn the MDP and **interleave acting and planning**.
- After each experience, update probabilities and the reward, then do some steps of **asynchronous value iteration.**

Model-based learner

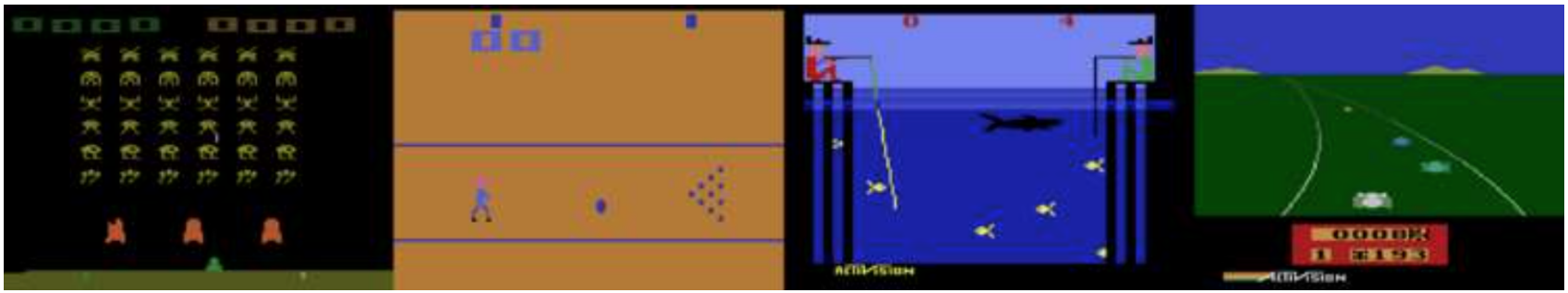
Data Structures: $Q[S, A]$, $T[S, A, S]$, $R[S, A]$
Assign Q, R arbitrarily, T = prior counts
α is learning rate
observe current state s
repeat forever:
  select and carry out action a
  observe reward r and state s'
  $T[s, a, s'] \leftarrow T[s, a, s'] + 1$
  $R[s, a] \leftarrow \alpha \times r + (1 - \alpha) \times R[s, a]$
  repeat for a while (asynchronous VI):
    select state s_1, action a_1
    let $P = \sum_{s_2} T[s_1, a_1, s_2]$
    $Q[s_1, a_1] \leftarrow \sum_{s_2} \frac{T[s_1, a_1, s_2]}{P} \left(R[s_1, a_1] + \gamma \max_{a_2} Q[s_2, a_2]\right)$
  end-repeat
  $s \leftarrow s'$;
end-repeat

Large State Spaces

- **Computer Go**: 3^{361} *states*



- **Atari** Games $210 \times 160 \times 3$ dimensions (pixels)



Q-function Approximations

- Let $s = (x_1, x_2, \ldots, x_N)^T$ where x are **features**
- **Linear**

$$Q_{\mathbf{w}}(s, a) \approx \sum_i w_{ai} x_i$$

- **Non-linear** (e.g. neural network g)

$$Q_{\mathbf{w}}(s, a) \approx g(\mathbf{x}; \mathbf{w_a})$$

Recall: Logistic Regression

Logistic function of linear weighted inputs:

$$\hat{Y}^{\overline{w}}(e) = f(w_0 + w_1 X_1(e) + \cdots + w_n X_n(e)) = f\left(\sum_{i=0}^{n} w_i X_i(e)\right)$$

The sum of squares error is:

$$Error(E, \overline{w}) = \sum_{e \in E} \left[Y(e) - f\left(\sum_{i=0}^{n} w_i X_i(e)\right)\right]^2$$

The partial derivative with respect to weight w_i is:

$$\frac{\partial Error(E, \overline{w})}{\partial w_i} = -2\delta f'\left(\sum_i w_i X_i(e)\right) X_i(e)$$

where $\delta = (Y(e) - f(\sum_{i=0}^{n} w_i X_i(e)))$.

Thus, each example e updates each weight w_i by

$$w_i \leftarrow w_i + \eta\delta f'\left(\sum_i w_i X_i(e)\right) X_i(e)$$

Approximating the Q-function

- assign weights $\mathbf{w} = \overline{\mathbf{w}} = \langle w_0, \ldots, w_n \rangle$ arbitrarily
- for experience tuple s, a, r, s', a' we have:
  - target Q-function: $[R(s) + \gamma \max_a Q_{\overline{\mathbf{w}}}(s', a)]$ or $[R(s) + \gamma Q_{\overline{\mathbf{w}}}(s', a')]$
  - current Q-function: $Q_{\mathbf{w}}(s, a)$
- Squared error:

$$Err(\mathbf{w}) = \frac{1}{2}\left[Q_{\mathbf{w}}(s, a) - R(s) - \gamma \max_{a'} Q_{\overline{\mathbf{w}}}(s', a')\right]^2$$

- Gradient:

$$\frac{\partial Err}{\partial \mathbf{w}} = \left[Q_{\mathbf{w}}(s, a) - R(s) - \gamma \max_{a'} Q_{\overline{\mathbf{w}}}(s', a')\right] \frac{\partial Q_{\mathbf{w}(s,a)}}{\partial \mathbf{w}}$$

- Weight update:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial Err}{\partial \mathbf{w}}$$

- Occasionally:

$$\overline{\mathbf{w}} \leftarrow \mathbf{w}$$

SARSA with linear function approximation

Given γ:discount factor; α:learning rate
Assign weights $\overline{w} = \langle w_0, \ldots, w_n \rangle$ arbitrarily
begin
observe current state s
select action a
repeat forever:
carry out action a
observe reward r and state s'
select action a' (using a policy based on $Q_{\overline{\mathbf{w}}}$)
let $\delta = r + \gamma Q_{\overline{\mathbf{w}}}(s', a') - Q_{\mathbf{w}}(s, a)$
For $i = 0$ to n
$w_i \leftarrow w_i + \alpha \times \delta \times \frac{\partial Q_{\mathbf{w}(s,a)}}{\partial w_i}$
$s \leftarrow s'$; $a \leftarrow a'$; (sometimes: $\overline{\mathbf{w}} \leftarrow \mathbf{w}$;)
end-repeat
end

Convergence

- Linear Q-learning ($Q_{\mathbf{w}}(s,a) \approx \sum_i w_{ai} x_i$) converges under same conditions as Q-learning

$$w_i \leftarrow w_i + \alpha \left[Q_{\mathbf{w}}(s,a) - R(s) - \gamma Q_{\mathbf{w}}(s',a') \right] x_i$$

- Non-linear Q-learning (e.g. neural network, $Q_{\mathbf{w}}(s,a) \approx g(\mathbf{x};\mathbf{w})$) may diverge
  - Adjusting $\mathbf{w}$ to increase Q at (s,a) might introduce errors at nearby state-action pairs.

Mitigating Divergence

Two tricks used in practice:

1. **Experience Replay**
2. **Use two Q function** (two networks):
   - Q network (currently being updated)
   - Target network (occasionally updated)

Experience Replay

- **Idea:** Store previous experiences (s, a, r, s', a') in a buffer and sample a mini-batch of previous experiences at each step to learn by Q-learning
- **Breaks correlations** between successive updates (more stable learning)
- Few interactions with environment needed to converge (greater **data efficiency**)

Target Network

- **Idea**: use a separate target network that is **updated only periodically**
- target network has weights $\overline{\mathbf{w}}$ and computes $Q_{\overline{\mathbf{w}}}(s, a)$
- repeat for each (s, a, r, s', a') in **mini-batch**:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[Q_{\mathbf{w}}(s, a) - R(s) - \gamma Q_{\overline{\mathbf{w}}}(s', a') \right] \frac{\partial Q_{\mathbf{w}}(s, a)}{\partial \mathbf{w}}$$

- $\overline{\mathbf{w}} \leftarrow \mathbf{w}$

Deep Q-Network

Assign weights $\overline{w} = \langle w_0, \ldots, w_n \rangle$ at random in $[-1, 1]$

begin

observe current state s

select action a

repeat forever:

carry out action a

observe reward r and state s'

select action a' (using a policy based on $Q_{\overline{\mathbf{w}}}$)

add (s, a, r, s', a') to experience buffer

Sample mini-batch of experiences from buffer

For each experience $(\hat{s}, \hat{a}, \hat{r}, \hat{s'}, \hat{a'})$ in mini-batch:

let $\delta = \hat{r} + \gamma Q_{\overline{\mathbf{w}}}(\hat{s'}, \hat{a'}) - Q_{\mathbf{w}}(\hat{s}, \hat{a})$

$\mathbf{w} \leftarrow \mathbf{w} + \alpha \times \delta \times \frac{\partial Q_{\mathbf{w}}(\hat{s}, \hat{a})}{\partial \mathbf{w}}$

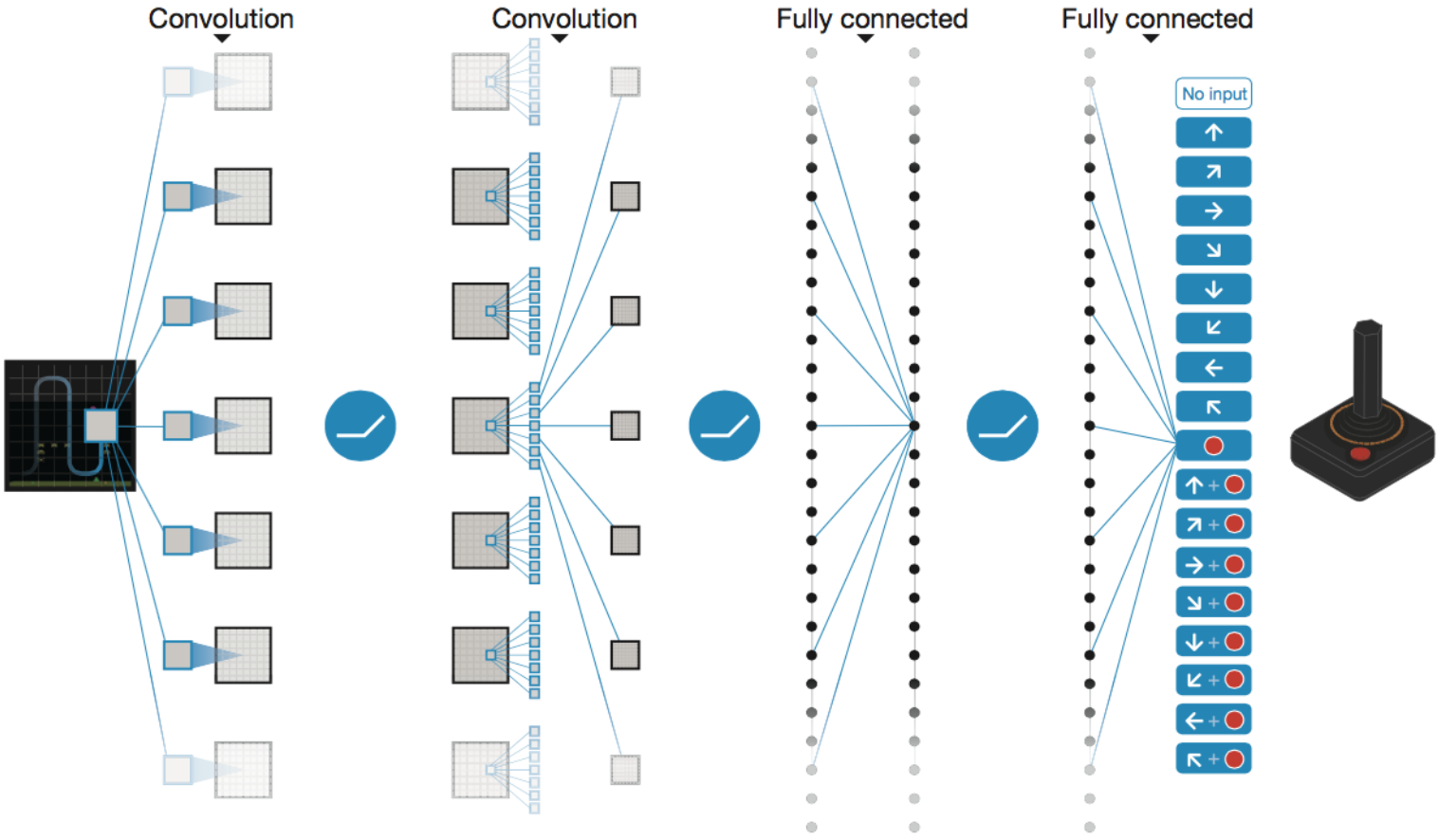
$s \leftarrow s'$; $a \leftarrow a'$;

every c steps, update target $\overline{\mathbf{w}} \leftarrow \mathbf{w}$

end-repeat

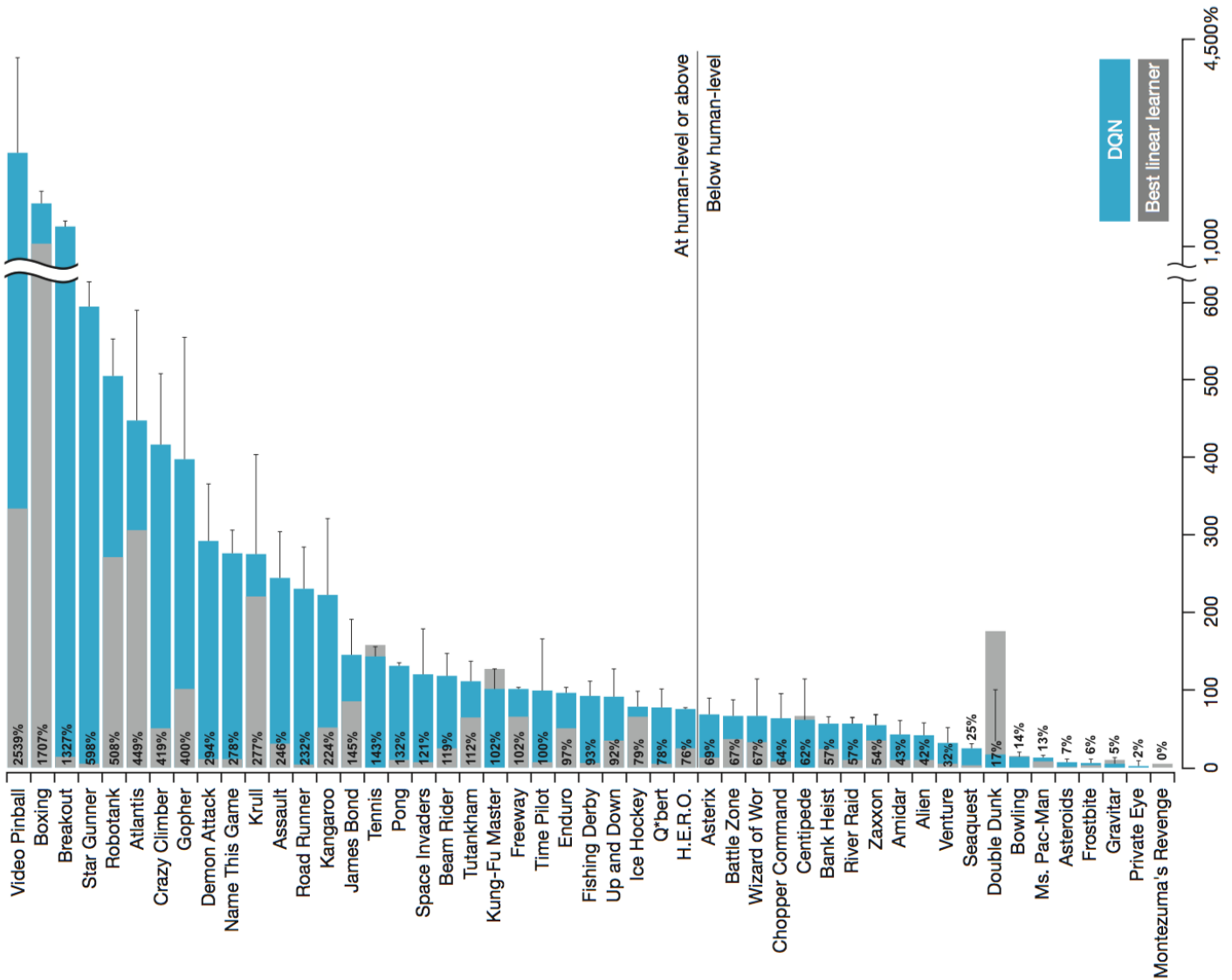
end

Deep Q-Network for Atari



from: Mnih et. al. Human-level control through deep reinforcement learning. Nature 18(7540):529–533 2015.

Deep Q-Network vs. Linear Approx.



Policy search using policy gradient

- Instead of approximating Q, we can also approximate policy π
- $\pi(a|s)$ is now a function that maps state s to a **distribution** over actions a
- Let $s = (x_1, x_2, \ldots, x_N)^T$ where x are **features**
- **Linear**

$$Q_{\mathbf{w}}(s, a) \approx \sum_i w_{ai} x_i$$

$$\pi_\theta(a|s) \approx \sum_i \theta_{ai} x_i$$

- **Non-linear** (e.g. neural network g)

$$Q_{\mathbf{w}}(s, a) \approx g(\mathbf{x}; \mathbf{w_a})$$

$$\pi_\theta(a|s) \approx g(\mathbf{x}; \theta_a)$$

Policy search using policy gradient

- How do we **change parameters θ of policy π_θ** to maximize objective?
- What is objective?
- V, Q: be value functions under π_θ
- $d^\pi(s) = \lim_{t\to\infty} P(s_t = s|s_0, \pi_\theta)$: be **stationary distribution of Markov chain** for π_θ
  - Imagine traveling Markov chain's states forever, eventually probability of ending up with one state becomes unchanged
- For more see:
  `lilianweng.github.io/posts/2018-04-08-policy-gradient`
- Want to maximize the objective:

$$J(\theta) = \sum_{s\in S} d^\pi(s) V(s) = \sum_{s\in S} d^\pi(s) \sum_{a\in A} \pi_\theta(a|s) Q(s,a)$$

How to find a good policy?

- Gradient is tricky because depends on both action selection π_θ and state distribution $d(s)$
- But updating policy changes state distribution!
- Policy gradient theorem (Sutton & Barto 2017; optional) allows us to simplify gradient

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \sum_{s \in S} d^\pi(s) \sum_{a \in A} \pi_\theta(a|s) Q(s,a) \\
&\approx \sum_{s \in S} d^\pi(s) \sum_{a \in A} Q(s,a) \nabla_\theta \pi_\theta(a|s) \\
&= \sum_{s \in S, a \in A} d^\pi(s) \pi_\theta(a|s) Q(s,a) \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} \\
&= \mathbb{E}_\pi \left[Q(s,a) \nabla_\theta \ln \pi_\theta(a|s) \right]
\end{aligned}
$$

where $\mathbb{E}_\pi$ refers to $\mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta}$ where both state and action distributions follow the policy (on policy).

REINFORCE algorithm (Williams 1992)

Use sampled trajectories to compute expectation over $Q(s, a)$

$$\begin{aligned}\nabla_\theta J(\theta) &\approx \mathbb{E}_\pi \left[Q(s,a) \nabla_\theta \ln \pi_\theta(a|s) \right] \\ &= \mathbb{E}_\pi \left[\left(\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \right) \nabla_\theta \ln \pi_\theta(a_t|s_t) \right]\end{aligned}$$

Initialize θ to random
Sample one trajectory using π_θ: s_1, a_1, r_2, s_2, a_2 $\ldots s_T$
For $t = 1, 2 \ldots T$
  Estimate $G_t = \sum_{k=0}^{T-t-2} \gamma^k r_{t+k+1}$
  Update $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_\theta \ln \pi_\theta(a_t|s_t)$
end

REINFORCE algorithm (Williams 1992)

- REINFORCE sampling trajectories to estimate gradient results in high variance
- Idea: subtract a baseline value from return G_t to reduce variance of gradient estimation
- Use **Advantage** $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$ instead of G_t

Actor Critic algorithm (Degris, White, & Sutton 2012)

- AC learns **two models, Actor π_θ and Critic Q_w**
- α and β are learning rates
- Use **Actor** to update $\pi_\theta(a|s)$ in the direction suggested by critic

Initialize s, w, θ to random
Sample $a \sim \pi_\theta(a|s)$
For $t = 1, 2 \ldots T$
  Sample reward $r_t \sim R(s, a)$, next state $s' \sim P(s'|s, a)$
  Sample next action $a' \sim \pi_\theta(a', s')$
  Update $\theta \leftarrow \theta + \alpha Q_w(s, a) \nabla_\theta \ln \pi_\theta(a|s)$
  Compute TD correction $\delta_t = r_t + \gamma Q_w(s', a') - Q_w(s, a)$
  Update $w \leftarrow w + \beta \delta_t \nabla_w Q_w$
  Update $a \leftarrow a'$, $s \leftarrow s'$
end

Bayesian Reinforcement Learning

- **Include the parameters** (transition function and observation function) in the state space
- **Model-based learning** though inference (belief state)
- State space is now continuous, **belief space is a space of continuous functions**
- Can mitigate complexity by **modeling reachable beliefs**
- **optimal** exploration-exploitation tradeoff.

Next:

- Recap of class + final review
- Research highlight
  - Jesse: Active inference and free energy
  - Victor: History of natural language processing